RESEARCH-ARTICLE

RICH Prefetcher: Storing Rich Information in Memory to Trade Capacity and Bandwidth for Latency Hiding

NINGZHI AI, Tsinghua University, Beijing, China

WENJIAN HE, Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

HU HE, Tsinghua University, Beijing, China

JING XIA, Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

HENG LIAO, Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

GUOWEI ZHANG, Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

Open Access Support provided by:

Tsinghua University

Huawei Technologies Co., Ltd.

Published: 18 October 2025

Citation in BibTeX format

MICRO 2025: 58th IEEE/ACM International Symposium on Microarchitecture
October 18 - 22, 2025
Seoul, Korea

Conference Sponsors:
SIGMICRO

https://doi.org/10.1145/3725843.3756081
ISBN: 9798400715730

RICH Prefetcher: Storing Rich Information in Memory to Trade Capacity and Bandwidth for Latency Hiding

Ningzhi Ai*
Huawei Technologies Co., Ltd
Shanghai, China
Tsinghua University
Beijing, China
anz22@tsinghua.org.cn

Wenjian He
Huawei Technologies Co., Ltd
Shanghai, China
hewenjian@hisilicon.com

Hu He
Tsinghua University
Beijing, China
hehu@tsinghua.edu.cn

Jing Xia
Huawei Technologies Co., Ltd
Shenzhen, China
dio.xia@hisilicon.com

Heng Liao
Huawei Technologies Co., Ltd
Shanghai, China
liao.heng@hisilicon.com

Guowei Zhang
Huawei Technologies Co., Ltd
Shanghai, China
zhangguowei9@hisilicon.com

Abstract

Memory systems characterized by high bandwidth and/or capacity alongside high access latency are becoming increasingly critical. This trend can be observed both at the device level—for instance, in non-volatile memory—and at the system level, as seen in CXL-based memory pooling architectures. To benefit from such memory in general-purpose computing systems, it is essential to employ techniques that can tolerate high memory access latency. Although prefetching has long been recognized as a classical approach for latency tolerance, conventional prefetching techniques are typically either optimized for area efficiency or constrained by limited prefetching patterns. Consequently, they often fail to convert the abundant metadata into significant performance improvements at minimal cost.

To address these challenges, we propose RICH—a prefetcher that strategically consumes memory capacity and bandwidth to reduce memory access latency. First, RICH is capable of leveraging abundant metadata to improve performance by integrating spatial prefetching with diverse region sizes and prefetch triggers. Second, RICH implements such metadata with minimal overheads by employing a hierarchical on-chip/off-chip storage mechanism, thereby avoiding both large on-chip storage and critical off-chip accesses. We propose a specific implementation of RICH and evaluate it across a wide range of workloads. With increased memory latency, RICH achieves performance improvements of 8.3% over Bingo and 6.2% over PMP. This highlights the RICH's suitability for future memory systems. In a conventional system, RICH still outperforms Bingo by 3.4%.

*Work done at Huawei.

MICRO '25, Seoul, Republic of Korea

ACM ISBN 979-8-4007-1573-0/25/10
https://doi.org/10.1145/3725843.3756081

ACM Reference Format:
Ningzhi Ai, Wenjian He, Hu He, Jing Xia, Heng Liao, and Guowei Zhang. 2025. RICH Prefetcher: Storing Rich Information in Memory to Trade Capacity and Bandwidth for Latency Hiding. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25), October 18–22, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3725843.3756081

1 Introduction

The skyrocketing memory demands [1, 2] and escalating memory costs [3, 4] have raised the importance of memory in modern infrastructure. However, as semiconductor technology node advances, traditional device shrinking suffers from diminishing returns and may no longer deliver Pareto improvements to concurrently achieve low latency, high bandwidth, and high capacity. Consequently, substantial research efforts have been directed towards **enhancing memory capacity and bandwidth**, albeit at the expense of **increased access latency**. At the device level, alternatives to DRAM, such as non-volatile memories (NVM) [5–7], typically offer much higher density. This strength can be converted to higher bandwidth or larger capacity, yet they inherently suffer from higher access latency [8]. Similarly, at the system level, methods like CXL-based memory pooling [9, 10] expand the accessible memory capacity, but again incur additional latency.

Although the trends in the memory industry align with the growing demands of AI and HPC workloads, they present significant challenges for general-purpose computing. Prefetching is a widely used technique designed to mitigate the penalty caused by memory latency. However, traditional prefetching mechanisms have not fully exploited the potential of high-capacity, high-bandwidth off-chip memory. Most prefetchers focus on minimizing hardware area and store data on-chip, which limits their ability to capture a wide range of memory access patterns. For example, prefetchers such as SMS [11], Bingo [12], and PMP [13] are restricted to predicting memory accesses within a 4 KB range. In contrast, some temporal prefetchers [14–16] can store long sequences of cache misses and take advantage of off-chip memory; however, these methods are only effective for applications whose memory accesses exhibit exact address sequence repetition.

In view of the aforementioned trends, we shift the direction of hardware prefetcher design to adopting more metadata, as oppose to confining metadata size. That is because the high-capacity high-bandwidth memory can accommodate and serve more information, while the increasingly critical memory latency issue requires a more performant prefetcher. However, this brings two non-trivial challenges: **1) how to convert the abundant metadata to performance gains**. Simple scaling-up of existing prefetchers often faces the law of diminishing marginal returns [12, 15, 17, 18]. However, complex metadata does not necessarily offer benefits, especially when a proper design is absent. **2) how to implement the metadata with minimal overheads**. Although we can offload metadata to off-chip memory, the latency of fetching the metadata can overturn the benefits. Moreover, the on-chip area is still scarce and limited. Therefore, careful arrangement is required to coordinate the off-chip memory.

In this paper, we propose the RICH prefetcher to address these challenges. The RICH prefetcher manages a **rich** set of metadata in a system with **rich** provision of memory capacity and bandwidth. First, RICH exploits multi-scale spatial locality by using 2 KB, 4 KB, and 16 KB regions, and employs a coordinated arbitration mechanism that selects appropriate regions to optimize both timeliness and coverage. Large-region prefetching only activates after multi-offset validation, achieving high prediction accuracy for prefetching long streams. In contrast, small-region prefetching is triggered via lightweight trigger matching, prioritizing broad coverage over precision. Second, RICH examines its metadata along three dimensions—the access frequency, latency tolerance, and area overheads—and employs a hierarchical storage design to effectively reduce both on-chip area and off-chip latency overheads. Our key contributions are as follows:

(1) We combine the high-performance benefits of large regions and the low misprediction advantages of small regions by designing a multi-level trigger mechanism that dynamically selects appropriate prefetch regions, significantly improving both timeliness and coverage.
(2) We minimize overall overheads by conducting a detailed analysis of the metadata and leveraging the high capacity of off-chip memory alongside the low latency of on-chip memory.
(3) We thoroughly simulate the RICH prefetcher, demonstrating its significant advantages over the state-of-the-art Bingo prefetcher—RICH outperforms Bingo by 3.4%. Additionally, with an extra 120 ns of memory latency, RICH shows a 8.3% improvement over Bingo, highlighting its superior performance in high-latency systems.

2 Background

2.1 High-latency Trends in Memory Technology

Driven by the excessive demands of modern workloads, the evolution of memory technologies increasingly prioritizes bandwidth and capacity over access latency.

According to [19], vast majority of desktop/scientific applications may have experienced performance loss due to the increased latency of commonly-used dynamic random-access memory (DRAM). This trend is evident in the DDR standards summarized in Table 1, where the peak bandwidth and unit capacity have more than tripled over generations, while the access latency has increased by 20%~40%.

Table 1: DDR specifications across generations exhibit the high-bandwidth high-capacity and high-latency trends

DDR Gen.	Peak speed	Max. size	latency on row hit	latency on row miss
3 [27]	2133 MT/s	8Gb	10-15ns	20-30ns
4 [28]	3200 MT/s	16Gb	12-15ns	25-30ns
5 [29]	7200 MT/s	64Gb	14-18ns	28-36ns

Emerging non-volatile memory technologies are expected to reshape memory hierarchies but face inherent latency challenges [8]. For instance, resistive memory [20] and phase-change memory [21] scale well with small cell sizes, making them cost-effective to offer higher capacity and/or bandwidth to compete with DRAM. However, their slower access time necessitates architectural adaptations.

At the system level, modern integration techniques may also exacerbate effective memory access latency. The high-bandwidth memory (HBM) [22] is a type of 3D-stacked DRAM to increase the memory density and interface bandwidth. However, as noted in [23] and later verified on real products by [24, 25], HBM incurs 25 ns latency overheads compared to conventional DDR, likely due to increased circuit complexity [24]. In addition, interconnect innovations like CXL [9] enable memory pooling [10]. It expands effective memory capacity by sharing unused space between computing nodes. Yet it introduces substantially higher access latency compared to local DDR [26].

In summary, memory subsystems are evolving toward higher bandwidth and capacity at the expense of access latency. This trajectory underscores the need to reevaluate design paradigms for future general-purpose computing systems.

2.2 Hiding Latency by Prefetching

One way to hide memory access latency is to prefetch. Prefetching refers to the act of predicting subsequent memory accesses and fetching the required data ahead of the processor execution [30]. To achieve this, a hardware prefetcher typically needs to manage training metadata, learned access patterns and control metadata. The training metadata stores partially learned program characteristics, and the control metadata decides how the learned access patterns are prefetched. We assess the effectiveness of prefetchers using the following metrics:

Accuracy: Accuracy indicates the correctness of the prefetch actions performed by the prefetcher. It is defined as the ratio of useful prefetches to all prefetches issued. Here, a *useful prefetch* is counted when a prefechted cacheline is subsequently accessed by the processor, and it only counts once. *Useless prefetches* refer to the number of prefetched cachelines that are unfortunately evicted before any access from the processor.

Coverage: Coverage measures the prefetcher's ability to detect the access patterns of a program. It is defined as the proportional decrease in cache misses with prefetching compared to a no-prefetching baseline—a method that has also been employed in previous research [13, 31].

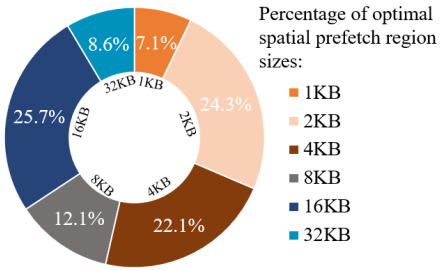


Figure 1: Different workloads favor distinct spatial region sizes for prefetching; approximately half achieve peak performance under conventional 4 KB regions, while the remainder achieve peak performance with larger spatial region sizes

Timeliness: Timeliness evaluates how well the prefetcher can anticipate cache misses and populate the cache in advance. Ideally, a prefetcher would eliminate all cache misses within the scope of its coverage. Timeliness is defined as the proportion of useful prefetches that have been filled into the cache by the time the processor accesses them, as opposed to remaining in the miss status handling registers (MSHR) .

Overheads: The overheads determine the practicality of a prefetcher. Most prefetchers focus on the area overheads, and therefore they usually constrain their metadata size.

2.3 Issues of Conventional Prefetchers

In view of the high-bandwidth, high-capacity memory trends, we believe it is essential to harness memory as an important component in prefetcher architectures. Therefore, we categorize hardware prefetchers into two groups: those who can leverage off-chip memory, and those who don't. We find that both groups of conventional prefetchers have not solved the challenges of how to leverage abundant metadata.

Existing prefetchers that can offload metadata into memory fail to convert the abundant metadata into performance gains for a wide range of workloads. Those prefetchers are mostly temporal prefetchers, which focus on irregular access patterns. For instance, ISB [14], MISB [15], and STMS [16] record streams of past cache misses and issue prefetches by replaying a stream when the identical leading access address appears again. Although they successfully out-source the storage of long streams and can tolerate high latency, temporal prefetchers can only support a specific class of applications. This is because they require exact access sequence repetition to work, which are only seen in programs with pointer-rich data references [32, 33] or complex instruction control flow [34–38].

Other prefetchers are not designed for the new challenges and restrict their metadata size. For instance, the next-line prefetcher and stride prefetchers [39, 40] can detect sequential accesses with small area overheads, but they are not effective for modern workloads with complex data structures [41, 42]. Offset-based prefetchers like BOF [43] can adapt to multiple strides, but they also suffer from insufficient metadata to separate different access patterns [41]. Likewise, existing spatial prefetchers focus on area-efficiency as well, and they are typically restricted to predict within a small memory region under 4 KB [11–13, 17, 44, 45].

2.4 Spatial Prefetcher Basics

In this work, we enhance spatial prefetchers with high-bandwidth, high-capacity memory to mitigate high memory latency.

Spatial prefetchers have a remarkable strength of eliminating compulsory cache misses [41]. Compulsory cache misses are a major source of performance degradation in important classes of applications [12]. To predict these misses, spatial prefetchers try to learn spatial correlation in data accesses. An application exhibits spatial correlation because its data objects or groups of data objects often share a same data structure and the same memory layout thereof. When accessing them, the program usually visits similar members of the objects. This correlation can be stable across previously unseen and distant memory locations, thereby eliminating compulsory cache misses.

The bit-vector-based prefetching represents an important form of spatial data prefetchers. It learns access patterns of memory regions at the granularity of a fixed size, e.g. 4 KB pages. Each pattern is encoded as a bit-vector, where each bit indicates a frequently visited cache block within the region. These learned patterns are typically stored in a set-associative storage known as the pattern history table (PHT). SMS [11] differentiates patterns based on the program counter (PC) of the load instruction together with the first access offset within the region, denoted as (PC, offset). Bingo [12] introduces a new trigger (PC, address) to partly cover temporal access patterns, where the address means the exact access address.

3 RICH: Design Philosophy

The design of our RICH prefetcher is centered on two key aspects. First, we identify the opportunity arising from the memory technology trends and leverage more predictor metadata to boost prefetching performance. Second, we address the challenge of minimizing the overheads of the new metadata by wise use of off-chip and on-chip resources altogether. The experiments of Section 3 are conducted using the SMS prefetcher [11] at L2 cache level with a system setup and benchmarks described in Section 5.1.

3.1 Synergy with Enriched Predictor Metadata

Insight 1

Workloads prefer diverse region sizes for spatial prefetching: e.g., half of them work best under conventional 4 KB sizes, while the other half prefer larger ones. Therefore, supporting multiple region sizes is crucial for performance.

As illustrated in Figure 1, different traces reach their highest performance at different region sizes, with 46% of the samples peak when the region size is set larger than 4 KB.

However, most spatial prefetchers capture memory access patterns within fixed 4 KB regions [11–13], which introduces two key limitations: First, spatial locality is not confined to a single scale but exhibits inherent diversity. Relying solely on a fixed prefetch granularity undermines the dynamic nature of spatial locality, resulting in limited coverage. Second, small regions are unable to fully exploit high bandwidth, which restricts the prefetcher's ability to issue long prefetch streams and degrades timeliness.

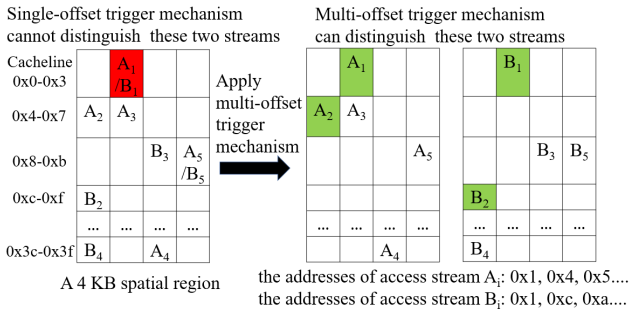


Figure 2: The multi-offset trigger mechanism can distinguish "common-origin, later-divergent" access streams

This finding motivates us to extend the range of regions captured by our spatial prefetcher, thereby accommodating a broader spectrum of program characteristics and enhancing performance. In our design, we support region sizes of 2 KB, 4 KB, and 16 KB, as they collectively ensure broad applicability.

Insight 2

Observing more access offsets within a spatial region before a prefetch trigger improves the accuracy at the cost of coverage.

It is less effective to use a traditional trigger method for 16 KB-region prefetching. Conventional spatial prefetchers usually use (PC, offset) or (PC, address) to predict the entire access stream, which may be acceptable for small regions. However, when applying the same triggering method for large-region prefetching, our tests show that the (PC, offset) triggering method suffers from low accuracy of less than 50%. Moreover, the (PC, address) triggering method only captures temporal locality sequences featuring repeated full addresses, thus severely limiting its coverage.

The accuracy limitation stems from the branching diversity within spatial region's footprints. As illustrated in Figure 2, different access streams may share the same initial access offset, yet diverge significantly in their subsequent trajectories. Conventional prefetchers only use the information of the first access, lacking the ability to validate the continuity of access patterns. Consequently, they are unable to distinguish these "common-origin, later-divergent" trajectories, leading to a dramatic increase in misprefetches.

To overcome this limitation, we propose to leverage multiple access offsets to predict the direction of an access stream. This strategy enables the prefetcher to differentiate between patterns that initially appear similar but later diverge. As illustrated on the right side of Figure 2, utilizing two offsets is sufficient to distinguish these streams. We further evaluate this approach by examining the trigger events ranging from (PC, 1 offset) to (PC, 10 offsets) for 16 KB-region prefetching, as shown in Figure 3a. The results reveal a significant upward trend in accuracy, strongly supporting the effectiveness of the multi-offset triggering method in capturing large-region access patterns. On the other hand, this method makes it harder to meet the stricter trigger condition, as we see the reduction of coverage in this figure. Note multi-offset triggering is applied in Figure 1.

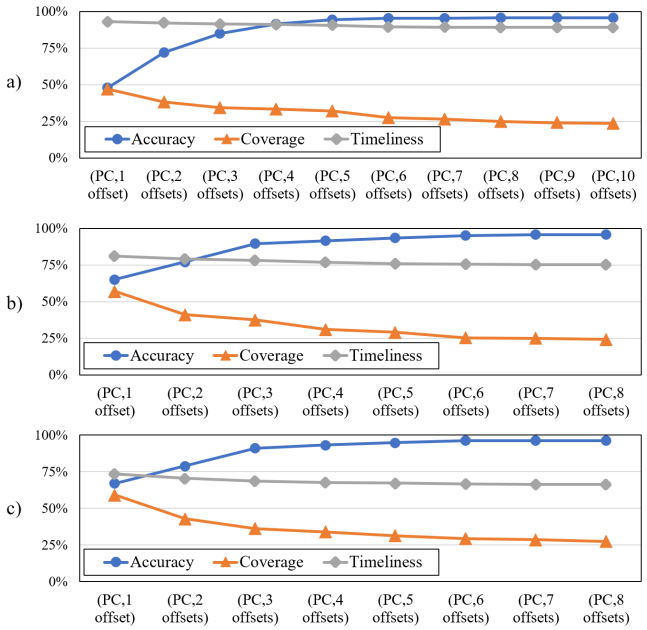


Figure 3: By using more access offsets in a trigger, we can trade off between the accuracy and coverage of prefetching. Specifically, a), b), and c) correspond to 16 KB-region, 4 KB-region, and 2 KB-region.

Insight 3

Large and small regions can be combined by leveraging different access offsets in prefetch triggers. This allows benefiting from the high performance gain of large-region prefetching and the low misprediction penalties of small-region prefetching.

Large-region prefetching and small-region prefetching offer different advantages. As shown in Figure 3, prefetching with the 16 KB-region provides significantly better timeliness compared to 2 KB-region and 4 KB-region, thereby delivering higher performance gains. However, this benefit comes with a major drawback: an incorrect decision incurs a significant penalty, as it fetches numerous useless cache lines, thereby wasting bandwidth and polluting the cache. In contrast, small-region prefetching has the advantage of a lower cost for mispredictions, which allows for more frequent triggering to improve coverage.

To capitalize on these complementary strengths, we implement an accuracy-first strategy for large-region prefetching while adopting a coverage-first strategy for small-region prefetching. This approach achieves synergistic optimization by aggressively exploiting prefetch opportunities in smaller regions, while constraining prefetches in larger region through the multi-offset trigger method.

In our analysis (see Figure 3a), the 16KB-region triggered by (PC, 5 offsets) achieves an accuracy of over 90%. While further increasing the number of access offsets leads to saturation in accuracy, it causes an noticeable drop in coverage. Therefore, we chose the (PC, 5 offsets) as the trigger to achieve the accuracy-first strategy. We apply the same approach and select (PC, 3 offsets) as the trigger for the 4 KB-region prefetching. For the 2 KB-region, we keep the

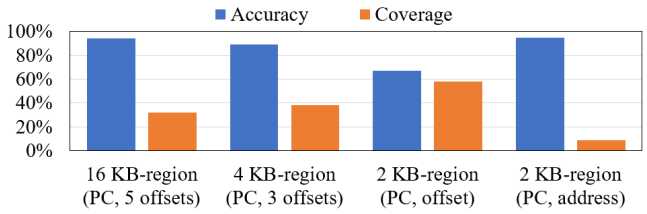


Figure 4: Achieving high accuracy in large spatial regions and high coverage in small spatial regions by using different numbers of trigger offsets

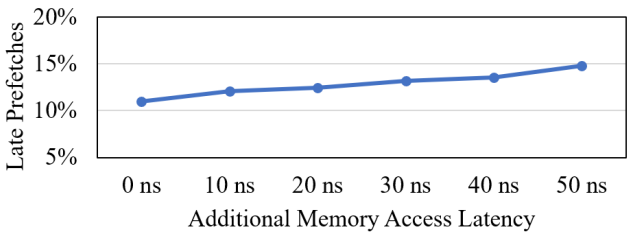


Figure 5: Placing 16 KB PHT off-chip keeps most prefetch opportunities, even with additional memory access latency

trigger method used by Bingo [12], as it provides sufficient coverage and captures aspects of temporal locality that compensate for the prefetcher's reliance on spatial locality.

In the case of multi-region prefetching, an important question arises: *if multiple regions are triggered simultaneously, how do we select the optimal region size to prefetch?*

In this work, we arrange their priority with an accuracy-first and performance-first approach. As shown in Figure 4, the 16 KB-region prefetches triggered by (PC, 5 offsets) achieve the highest accuracy, followed by the 4 KB prefetches, while the 2 KB prefetches triggered by (PC, offset) exhibits the lowest accuracy. This accuracy trend aligns with the performance trend of these regions. Ideally, correct prefetching of 16 KB regions yields best performance, followed by the correct prefetching of 4 KB regions, and 2 KB regions at last. This trend alignment allows us to assign a high priority to 16 KB-region prefetches, then 4 KB, and finally the 2 KB region prefetches triggered by (PC, offset). Notably, the 2 KB-region prefetching triggered by (PC, address) offers the comparable accuracy to that of the 16 KB. In view of its less misprediction cost, we give the highest priority to 2 KB-region prefetching triggered by (PC, address).

We further introduce a de-duplication mechanism to reduce prefetches on the overlapping memory space. For example, after the prefetching of a 16 KB region is triggered, subsequent prefetch triggers are not allowed within this region before this 16 KB prefetching finishes. Likewise, 4 KB prefetching can block 2 KB prefetching. This is based on the fact that larger regions have higher accuracy with the help of the multi-offset trigger mechanism. Notably, the (PC, 5 offsets) condition of 16 KB region prefetching requires at least five addresses to be observed before it can trigger, whereas 2 KB-region prefetches can be triggered by a single address. In this case, the 2 KB prefetching does not wait for the prediction of 16 KB regions, and 2 KB prefetches are initiated first. This mitigates potential prefetch opportunity losses due to the late prediction of large-region prefetches.

3.2 Synergy Between On-chip and Off-chip

To implement the above algorithm featuring enhanced prefetch patterns and prefetch triggers in minimal costs, we strategically partition the metadata between on-chip and off-chip. Storing all metadata on-chip would be expensive despite the low-latency advantage, while relying solely on off-chip leads to late prefetches.

Insight 4

The metadata exhibits distinct properties in terms of sizes, latency tolerance and access frequencies. This facilitates our partitioning to minimize the on-chip area and the penalty of off-chip accesses.

We categorize the metadata used by the RICH prefetcher in Table 2. Metadata that incur substantial area overheads yet tolerate high access latency can be assigned to off-chip storage, whereas those with smaller area requirements and high access frequency demands are better accommodated on-chip.

Metadata suitable for off-chip storage: We store infrequently accessed large-region patterns in main memory to minimize on-chip area overheads. Each 16 KB-region access pattern is encoded as a 256-bit vector. In our experiments, a single trace can contain over 3500 distinct patterns, and storing all of these vectors on-chip would incur over 100 KB of costly on-chip overheads. However, the 16 KB-region introduces a natural tolerance to access latency: a single prefetch request for such a large region provides a high volume of prefetches, ensuring ample prefetch opportunities even with delayed metadata retrieval. To quantify the impact of off-chip latency, we measure the loss in prefetch opportunities between the initiation of a 16 KB-region prefetch request and the arrival of its corresponding pattern. As demonstrated in Figure 5, even with an aggressive additional delay of 50 ns atop baseline memory access latency, prefetch opportunities degrade by less than 15%.

Moreover, in terms of access frequency, it is found that the indices generated for the large-region patterns are highly concentrated. This concentration indicates that a small set of pattern types can cover a large portion of prefetch requests. We conduct an experiment where the capacity of the 16 KB-region patterns is set to 200 entries. We track the percentage of prefetch requests that these patterns can cover. As summarized in Table 3 for the representative traces that largely benefit from 16 KB-region prefetching, the 200 on-chip patterns could cover a large portion of prefetch requests.

Considering that 16 KB-region patterns exhibit a mixed frequency distribution, substantial area overheads, and high latency tolerance, we propose a tiered storage strategy for these patterns. In this approach, a compact on-chip cache is dedicated to high-frequency patterns for rapid access, while low-frequency patterns are relegated to off-chip storage to minimize area overheads.

Furthermore, we also try to reduce the number of off-chip access requests. It is crucial to note that not every 16 KB-region pattern is

Table 2: Categorization of RICH's metadata

	Large region's patterns	Small region's patterns	Training metadata	Control metadata
Area overheads	high (> 100 KB)	medium (~ 30 KB)	low (< 10 KB)	low (< 10 KB)
Access frequency	mixed	mixed	high	high
Latency tolerance	high	low	low	low
RICH's structures	16 KB-region PHT Cache, 16 KB-region PHT	4 KB-region PHT, 2 KB-region PHT	FTs, ATs	Inflight Prefetch Table, 16 KB-region Valid Map
RICH's decision	Frequently accessed: on-chip, Infrequently accessed: off-chip	on-chip		

Table 3: Cover ratio under a 200-entry storage constraint of 16 KB-region patterns

Trace	% Covered	Trace	% Covered
bwaves-891	100.0%	bwaves-2609	80.5%
gcc-1850	93.7%	fotonik3d-7084	66.9%
mcf-782	90.4%	roms-1070	38.7%
mcf-1152	86.7%	roms-294	32.8%

worthwhile for off-chip load/write operations. The benefit of off-chip loading lies in the performance gains from prefetching a large number of cache lines; however, this benefit comes at the potential cost of performance degradation due to two possible row buffer misses during a metadata access (assuming sufficient bandwidth and neglecting any bandwidth loss). Given that future memory systems may exhibit higher access latency, the impact of row buffer misses are even more pronounced. Consequently, we resort to off-chip metadata accesses only when the performance gains outweigh the losses from row buffer misses. Specifically, the system triggers a 16 KB-region off-chip metadata write/load only when the prefetch count of the pattern, i.e. the number of '1' inside its bit-vector, exceeds a predefined threshold. This filtering mechanism prevents unnecessary off-chip operations, thus minimizing the number of off-chip access requests.

Metadata suitable for on-chip storage: We allocate the remaining components to on-chip storage. The patterns for small regions (2 KB and 4 KB) exhibit greater sensitivity to latency, making them unsuitable for off-chip storage. The prefetcher's training metadata is kept on-chip because it needs to interact frequently with the L2 cache to capture spatial patterns. Finally, the control metadata is also placed on-chip. The control metadata only occupies a small area and requires frequent decisions, thus is appropriate for on-chip storage. The control metadata will be detailed in the next section.

4 RICH: Design and Implementation

Equipped with the insights above, Figure 6 depicts the implementation of our RICH prefetcher. The RICH prefetcher is partitioned into the on-chip part and the off-chip part.

The on-chip part consists of three categories: the pattern training, the pattern storage, and the control components. The Training Unit captures access patterns by observing the accesses of the processor core upon L2 cache misses and L2 cache hits on prefetched cache lines. The PHTs store learned access patterns. Specially, the on-chip storage of 16 KB-region patterns acts like a cache for the off-chip part. To detect prefetch opportunities, the Prediction Unit

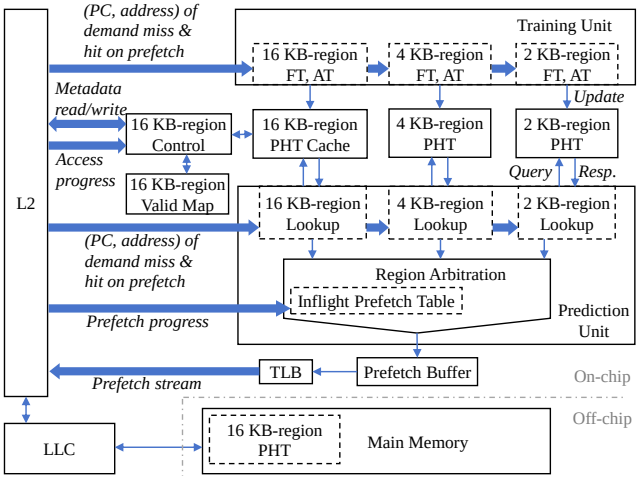


Figure 6: Architecture of RICH

consults the pattern storage with information of recent accesses from the processor core. The Region Arbitration controls whether the prefetch requests can proceed to the Prefetch Buffer and eventually to the cache. An additional control unit is introduced to manage metadata read/write for off-chip 16 KB-region patterns. As the 16 KB-region prefetching of RICH crosses 4 KB page boundaries, virtual addresses are used to ensure pattern continuity. Therefore, the prefetch transactions need go through address translation.

The off-chip part holds the PHT for 16 KB-region prefetching, which stores infrequently accessed 16 KB-region patterns. Upon the eviction of a 16 KB-region PHT Cache entry, it may be written to the main memory through the cache hierarchy.

Next, we will detail the training process, the on-chip prediction process and the off-chip prediction process.

4.1 Training Process

On-chip training process: Our training process builds upon the method of SMS [11], while RICH distinguishes itself by capturing memory access patterns using multiple offsets. The training unit consists of two tables for each region: the Filter Table (FT) and the Accumulation Table (AT). The FT filters out noisy patterns by identifying regions with insufficient access activity, while the AT accumulates memory access footprints to construct prefetch patterns. As shown in Figure 7, we use a (PC, 2 offsets) trigger as an example. (1). Here, a new region is accessed with an offset of 3. the FT logs the access without learning a full pattern. (2). A subsequent

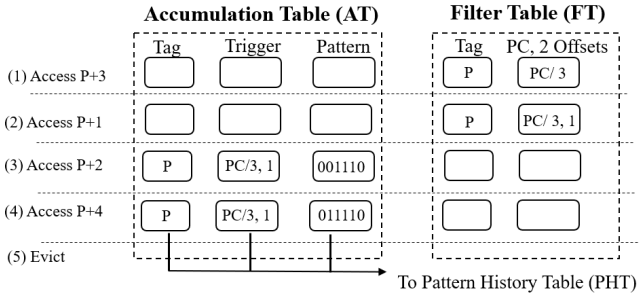


Figure 7: Training process of each region

distinct offset 1 is recorded. (3). Once an third different offset (here, 2) is observed, the FT entry is promoted to the AT, which initializes a bit-vector. (4). The AT then updates the bit-vector as more accesses occur. (5). When the cache starts evicting this region or the AT reaches capacity, the pattern is finalized and transferred to the PHT for prediction. The least-recently-used (LRU) replacement policy is employed by the three on-chip PHTs.

Off-chip training process: When the 16 KB-region On-chip PHT Cache evicts a pattern due to capacity overflows, the pattern can spill to the off-chip PHT. Note only those with sufficient prefetch counts are worthy to do so as discussed in Section 3.2. The off-chip PHT is an array data structure with the size of 4096 patterns. Each bit-vector pattern takes 32 bytes, therefore 128 KB in total. To speed up addressing, 12-bit trigger IDs are used to distinguish 16 KB patterns, i.e., at most 4096 trigger-pattern pairs. Therefore, we have a one-to-one mapping between trigger IDs and off-chip patterns, and only the on-chip PHT cache needs to store tags. To help the prediction process to know whether a valid pattern be fetched from off-chip, the Valid Map unit maintains an array of 4096 bits indexed by 12-bit trigger IDs. Whenever a pattern is written out to off-chip, the corresponding bit is updated to 1. The Valid Map is cleared periodically to invalidate stale off-chip patterns.

In our implementation, we use a prefetch count threshold of 30 according to experiments. And 128 KB main memory is used because it is sufficient to accommodate the 16 KB-region patterns for our benchmarks. In our design, the operating system allocates the main memory space for RICH at the boot time. After that, this space becomes dedicated to our prefetcher and inaccessible to applications. The base physical address of this PHT space will be configured via the system register of RICH.

4.2 On-chip Prediction Process

This process consists of 3 steps. First, the Lookup Units of 3 regions concurrently collect access offsets to form triggers. Second, upon a trigger, the corresponding on-chip PHT is searched. If hit, the prefetch request is sent to the Arbitration Unit before a prefetch stream can be generated. Specially, when the 16 KB-region encounters an on-chip PHT miss, it can enter the off-chip prediction process. Next, we explain the flow with Pseudo-code 1.

Step P1 (Line 13-24): Our multi-offset mechanism requires several accesses before prefetch trigger. For this, we use circular FIFO buffers to track recent accesses in the Lookup Units. As the 16-KB region uses the (PC, 5 offsets) trigger, its FIFOs have a capacity of 5. Upon a monitored access, its offset is pushed into a FIFO (Line 22).

Pseudo-code 1 The prediction process of RICH.

```
1:  def Do_Prediction_Process(pc, address):
2:      # Execute upon demand misses or hits on prefetch
3:      req16KB = Do_Lookup_16KB(pc, address)
4:      req4KB  = Do_Lookup_4KB(pc, address)
5:      req2KB  = Do_Lookup_2KB(pc, address)
6:      if req16KB == req4KB == req2KB == None: return;
7:      # Step P3: arbitrates between regions:
8:      result = Do_RegionArbitration(req16KB, req4KB, req2KB)
9:      if result.outcome != ignored:
10:         InflightPrefetchTable.insert(result.win_req)
11:         Generate_Prefetches(result.win_req)
12: def Do_Lookup_16KB(pc, address):
13:     global FIFOs16KB # circular FIFO buffers in Lookup unit.
14:     # 16KB has 2 FIFOs. Each FIFO tracks 1 region and
15:     # can hold at most 5 pc-offset pairs.
16:     # Step P1: accumulate observations for offsets
17:     region_tag = address / (16*1024)
18:     offset = (address % (16*1024)) >> 6
19:     if not FIFOs16KB.any_is_tracking(region_tag):
20:         randomly clear a FIFO, set it to track region_tag
21:     if offset not in FIFOs16KB[region_tag]:
22:         push (pc, offset) into FIFOs16KB[region_tag]
23:     if not FIFOs16KB[region_tag].full(): return None
24:     if TrainUnit16KB.is_learning(region_tag): return None
25:     # Step P2: after 5 offsets collected, query on-chip PHT
26:     trigger_id = hash(FIFOs16KB[region_tag]) # output 12 bits
27:     pattern = Onchip_PHT_Cache_16KB.find(trigger_id)
28:     if pattern.found(): return (region_tag, pattern)
29:     # If not found, 4 KB and 2 KB stops here and return None.
30:     # Step M: on-chip miss, go off-chip. (16 KB exclusive)
31:     offchip_idx = trigger_id # one-to-one mapping
32:     if ValidMapUnit[offchip_idx] == True:
33:         # This takes long time and runs asynchronously.
34:         ControlUnit16KB.Offchip_Metadata_Read(
35:             offchip_idx, callback=Do_RegionArbitration)
36:     return None
37: def Do_RegionArbitration(req16KB, req4KB, req2KB):
38:     win_req = FixedPriorityArb(req16KB, req4KB, req2KB)
39:     outcome = InflightPrefetchTable.has_overlap(win_req)
40:     return (outcome, win_req)
```

When 5 offsets are collected, it can proceed. Otherwise, this region turns idle (Line 23). As a program can work on multiple spatial space at the same time, we need to track offsets separately. Therefore, the 16-KB region uses 2 FIFOs, each tracks a particular 16 KB-aligned virtual space (Line 17). When the program touches another new region, a FIFO is reset to track this new space (Line 19-20). The 4 KB-region Lookup Unit also uses 2 FIFOs, each with a capacity of 3 for the (PC, 3 offsets) trigger. As for the 2 KB-region, it triggers with a single access, so FIFOs are unnecessary. At last, to prevent repeated triggers, we apply the same filter used in [11, 12] (Line 24).

Step P2 (Line 25-29): After sufficient offsets are collected in a FIFO, the prediction process proceeds to access the on-chip PHT. The trigger ID is calculated by a hash function to blend the oldest PC and all offsets stored in the FIFO (Line 26). If the on-chip PHT contains the pattern for this trigger (i.e. PHT hit), the pattern is forwarded to the Region Arbitration unit.

Step P3 (Line 7-11, 37-40): The region arbitration step chooses the best pattern to prefetch for the triggered memory space. This choice is required in two aspects: (1) when two or more regions simultaneously hit PHTs; (2) when a prefetch decision made in the past overlaps with the region triggered at present. We solve these two aspects with 2 levels of arbitration as depicted in Figure 8.

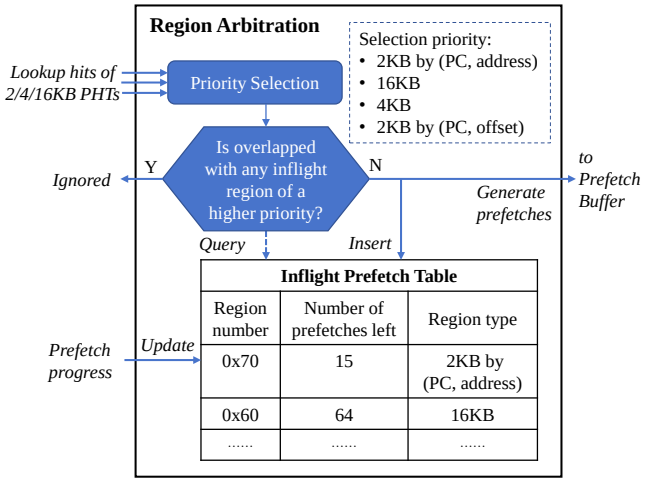


Figure 8: The Region Arbitration unit selects the appropriate region to prefetch (Step P3)

For (1), we use a fixed priority to pick a pattern among the PHT hits according to the region characteristics discussed in Section 3.1. The order is as follows: the 2 KB-region triggered by (PC, address) has the highest priority, followed by the 16 KB-region, then the 4 KB-region, and finally, the 2 KB-region triggered by (PC, offset).

For (2), the winning request of (1) is subsequently checked with past prefetches recorded in the the Inflight Prefetch Table. If the current request overlaps with a former prefetch decision, the current one must come from a higher priority to generate prefetches. The priority order is the same as (1) above. The number of inflight prefetches determines the lifespan of a table entry. Every time a cacheline is prefetched to the L2 cache, the corresponding numbers in the Inflight Prefetch Table are decremented. After it reaches zero, the entry is retired.

Additionally, when the 16 KB-region fails to hit in the on-chip PHT cache, it can enter off-chip prediction process discussed below.

4.3 Off-chip Prediction Process

The purpose of the off-chip prediction is to retrieve the 16 KB-region pattern stored in main memory and generate a prefetch stream. It involves the 16 KB-region PHT Cache, 16 KB-region Valid Map, the off-chip 16 KB-region PHT and the 16 KB-region Control. As shown in Step M of Pseudo-code 1 (Line 30-35), the process is entered when a trigger misses in the 16 KB-region On-chip PHT Cache and the corresponding bit in the Valid Map is 1, indicating that the pattern resides in the main memory.

As a memory access can take a long time, this process runs asynchronously. We describe the process in three sub-steps. First, a metadata load request is issued to read the off-chip PHT.

Second, the 16 KB-region Control unit monitors the processor to track late prefetches. Because there is a time gap between issuing the metadata load and receiving the pattern from the main memory. During this interval, the processor may have accessed some parts within the region to prefetch. If we still issue prefetches for them, they would result in useless prefetches and can harm performance. The late prefetches are tracked at a 2 KB sub-region granularity, so 8 bits for a pending 16 KB region.

Table 4: Breakdown of on-chip storage overheads

Structure	Width (bits)	Size (entries)	Storage (KB)
FTs (2, 4, 16 KB)	64, 64, 90	64, 64, 64	1.7
ATs (2, 4, 16 KB)	91, 106, 312	128, 128, 32	4.3
PHTs (2, 4 KB)	43, 71	4096, 1024	30.4
16 KB-region PHT Cache	271	256	8.5
Inflight Prefetch Table	48	6	0.05
16 KB-region Valid Map	1	4096	0.5
Prefetch Buffer	48	320	1.9
Total			**47.3**

Third, once the pattern is returned, it is saved to the On-chip PHT Cache, and at the same time, it is sent to Region Arbitration after being trimmed by the marked 2 KB sub-regions of late prefetches. As this happens asynchronously, it is equivalent to execute a `Do_RegionArbitration(req16KB, None, None)` call as defined in the Pseudo-code. Based on the arbitration result, the final prefetch stream can be generated.

4.4 Storage Overheads

Table 4 lists the storage overhead breakdown of RICH. The number of PHT entries in the 2 KB-region exceeds that of the 4 KB-region because the (PC, address) trigger event generates a much larger variety of entries compared to the (PC, 2 offsets) trigger event. Nonetheless, we still significantly reduced the 16,384 entries required by Bingo. As we discussed earlier, the 16 KB-region PHT only holds a small 256-entry PHT on-chip. We observe that a 6-entry Inflight Prefetch Table is sufficient to track the inflight prefetch regions. We set the depth of Prefetch Buffer to 320 to allow for aggressive prefetches. Compared to the Bingo's requirement of 127 KB, we have significantly reduced the on-chip area. As for off-chip, we occupy 128 KB of main memory by each core, which is very small in current systems equipped with gigabytes of main memory. We use CACTI [46] with its 22 nm configuration to estimate the area consumption and the cache access time of our design. The area of our three on-chip PHTs is 0.21 mm², the total power consumption is 55 mW/core, and the longest access time is 0.39 ns for indexing the 2 KB-region PHT.

5 Evaluation

5.1 Methodology

1) Configuration: We evaluate RICH using the ChampSim simulator [47, 48], a trace-driven, cycle-accurate simulator that is capable of modeling modern out-of-order processors. ChampSim has also been utilized in the second and third Data Prefetching Championships (DPC-2 [49] and DPC-3 [50]), the second Cache Replacement Championship (CRC-2) [51]. In our experiments, we model an Intel Alder Lake performance core [52]. The configuration parameters are obtained from public sources [53–56].

2) Prefetchers: We compare RICH against four spatial prefetchers: Bingo [12], PMP [13], SPP-PPF [57], and SMS [11]. Bingo is

Table 5: Simulated system

Core	8-wide fetch, 6-wide decode/dispatch, 512-entry ROB, 192/114-entry LQ/SQ, Perceptron branch predictor with 17-cycle misprediction penalty
L1-I Cache	Private, 32KB (8-way, 64B line), 4-cycle round-trip latency, 8 MSHRs
L1-D Cache	Private, 48KB (12-way, 64B line), 5-cycle round-trip latency, 16 MSHRs
L2 Cache	Private, 1.25MB (20-way, 64B line), 15-cycle round-trip latency, 48 MSHRs
LLC	3MB/core (12-way, 64B line), 55-cycle round-trip latency, 64 MSHRs
Main Memory	1C: Single channel, 1 rank/channel; 4C: Dual channel, 2 ranks/channel. DDR5-4800 MTPS, 64-bit data bus per channel, tRCD=tRP=tCAS=16.6ns

Table 6: Comparison of area overheads

Bingo	PMP	SPP-PPF	SMS	RICH
127.8 KB	4.3 KB	48.4 KB	116.6 KB	47.3 KB on-chip, 128 KB off-chip

Table 7: Workloads used for evaluation

Suite	Count	Example workloads
SPEC06	46	gcc, bwaves, mcf, leslie3d, GemsFDTD
SPEC17	42	gcc, bwaves, mcf, cactuBSSN, lbm
Ligra	40	Bellman-ford, Triangle, Radii, PageRank
Parsec	12	canneal, fluidanimate, streamcluster

one of the state-of-the-art spatial prefetchers.PMP, the latest light-weight prefetcher based on bit-pattern analysis, employs a strategy of merging similar patterns and achieves high coverage. SPP-PPF stands out among delta-based prefetchers in DPC-3. SMS, one of the earliest bit-vector-based spatial prefetchers, is configured with a 16k-entry PHT to achieve optimal performance. Table 6 summarizes the storage overheads associated with these prefetchers. All prefetchers are deployed at the L2 cache level, without helper prefetchers in other levels.

3) Workloads: We evaluate the single-core performance of the five prefetchers using 100+ traces collected from SPEC CPU 2006 [58], SPEC CPU 2017 [59], Ligra [60] and Parsec [61]. SPEC traces are from DPC-3 [62]. For Ligra and Parsec, we use the traces provided by Pythia [31, 63, 64]. We omit the traces whose L2 cache misses per thousand instructions (MPKI) are less than 2, as they are less impacted by the memory access latency. Table 7 lists the numbers of traces in our tests. We evaluate the multi-core performance of the five prefetchers using both homogeneous and heterogeneous workloads on a 4-core system. For homogeneous workloads, we assess prefetchers using all traces, where each trace is executed simultaneously by different cores. For heterogeneous workloads, we run 40 mixes, each consisting of four randomly selected traces. In both single-core and multi-core systems, We use the first 50 million instructions of a trace to warm up microarchitectural structures and the next 200 million instructions to examine the performance.

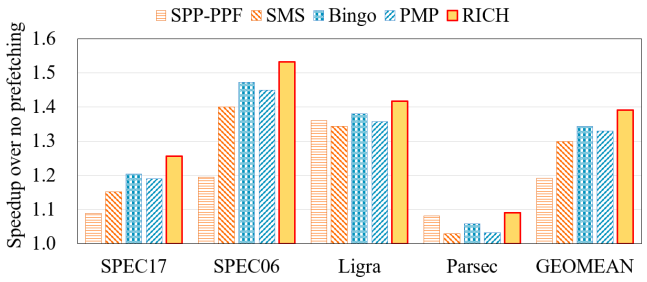


Figure 9: RICH outperforms Bingo across four benchmark suites on a single-core system

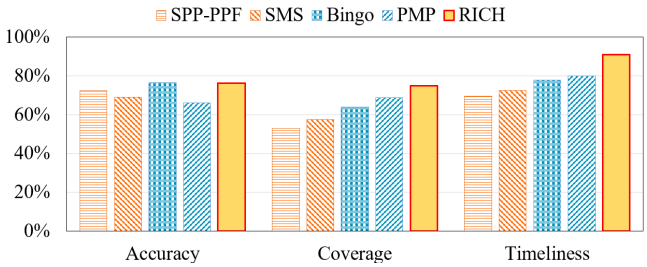


Figure 10: RICH improves coverage and timeliness while maintaining comparable accuracy

5.2 Single-core Performance

Figure 9 illustrates the speedup achieved by the evaluated prefetchers in a single-core system. RICH delivers the highest performance improvement across all workload suites, achieving an average improvement of 39% over a no-prefetching baseline. Specifically, RICH outperforms the state-of-the-art spatial prefetchers — Bingo by 3.4%, PMP by 4.4%, SMS by 6.6% and SPP-PPF by 16.8%.

Although SPP-PPF incorporates many filtering features for predictions generated by the Signature Path Prefetcher (SPP) [17], it does not fully exploit the high bandwidth resources, resulting in suboptimal performance. Meanwhile, SMS, the earliest bit-vector-based spatial prefetcher, is confined to capturing access patterns within fixed 2 KB regions using the (PC, offset) trigger event, and this limits its capacity to capture more patterns.

In contrast, the RICH prefetcher achieves greater coverage and timeliness relative to PMP and Bingo, while maintaining accuracy comparable to Bingo and surpassing PMP. Consequently, its overall performance exceeds that of both Bingo and PMP. Detailed comparisons of these three metrics will be provided below.

Figure 10 shows the average accuracy, coverage and timeliness metrics for all prefetchers. The RICH prefetcher achieves 11% higher coverage than Bingo (75% vs. 64%), 13% better timeliness (91% vs. 78%), and a comparable accuracy (76.3% vs. 76.6%). In comparison to PMP, RICH delivers 6.2% higher coverage, 11% higher timeliness, and 10% higher accuracy. RICH not only ensures high accuracy but also significantly improves coverage and timeliness, fundamentally due to its effective utilization of rich metadata to enhance performance. We further analyze the contribution of each region to overall coverage and the Average Miss Access Time. Figure 12a

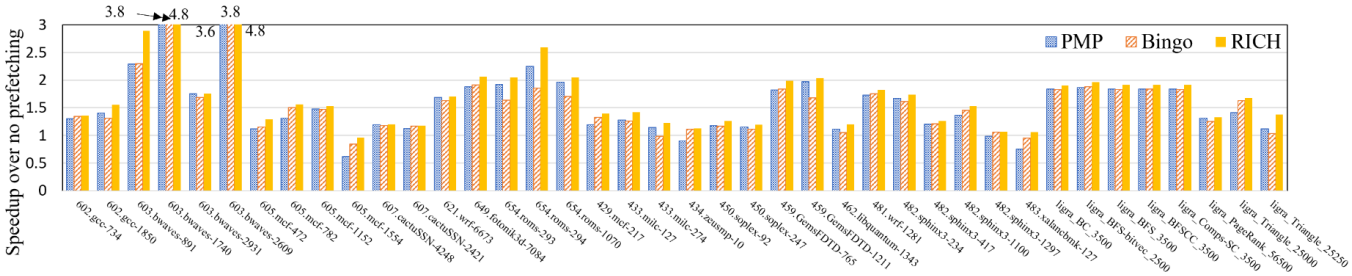


Figure 11: Detailed speedup achieved by Bingo, PMP and RICH on representative traces

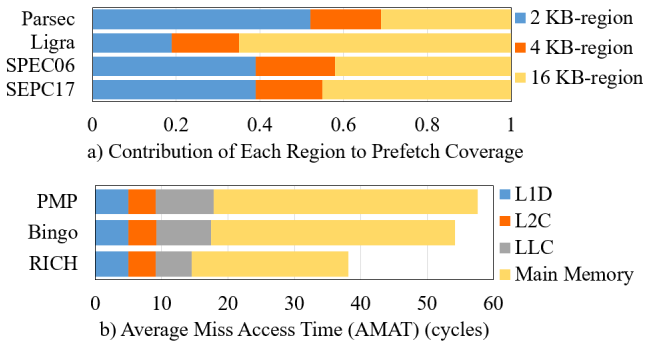


Figure 12: Each region makes a significant contribution to prefetching, and we achieve a significant reduction in AMAT

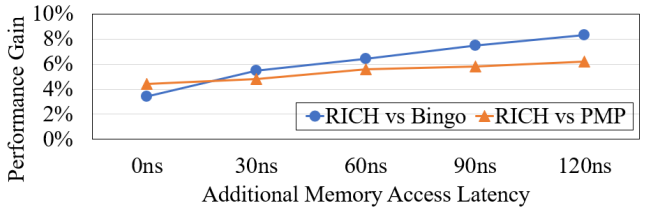


Figure 13: RICH demonstrates greater performance gains compared to Bingo and PMP as memory latency increases, highlighting its potential for future high-latency systems

shows that each region size plays a non-trivial role. Based on the hit rates across cache levels, we calculate the AMAT, as shown in Figure 12b, where RICH significantly reduces the total access time by up to 30% compared to Bingo.

In terms of coverage, RICH's superiority stems from its multi-region and multi-offset trigger mechanism, coupled with an advanced selection strategy. Together, these approaches enable RICH to overcome the limitations of conventional fixed granularity and effectively capture data objects of varying sizes, thereby accommodating a broader spectrum of memory access patterns.

In terms of timeliness, RICH has the advantage of prefetching a larger spatial region. Unlike SMS and Bingo, which can only prefetch within a 2 KB region, RICH can prefetch within 4 KB and 16 KB regions, significantly increasing the prefetch distance, thereby improving timeliness.

In terms of accuracy, RICH's advantage stems from its ability to use multiple offsets in a prefetch trigger, rather than relying solely on single-access information for triggering. While Bingo uses a triggering mechanism based on (PC, address) to ensure high accuracy within 2 KB, RICH can prefetch larger regions (4 KB and 16 KB) and still maintain high accuracy.

We enumerate some representative traces in Figure 11 and provide concrete examples to further illustrate RICH's advantages.

Case study on improving coverage: Our algorithm effectively identifies the multi-scale spatial locality of both large and small regions, selecting the appropriate prefetch granularity during execution. For instance, in the *459.GemsFDTD-765B*, dense accesses occur within 2 KB sub-regions, while inter-page spatial correlations span a 16 KB range. Traditional prefetchers, which rely on fixed small-region prefetching, struggle with spatial locality mismatches, resulting in only 0.54 coverage. RICH addresses this challenge by using multi-region prefetch, improving coverage to 0.78 and increasing useful prefetches by 45%. Additionally, huge prefetches were triggered for both 4 KB and 16 KB regions, with their accuracy reaching 0.92. Overall, performance improved from Bingo's 182% to 199%. This performance gain stems from our dynamic granularity switching selection mechanism, which effectively leverages multi-scale spatial locality.

Case Study on improving timeliness: Some traces such as *603.bwaves-891B*, *602.gcc-1850B* and *654.roms-1070B* exhibit a large amount of cross-4 KB spatial locality. RICH can seize these large-region spatial locality and utilize high-bandwidth memory to launch a large number of prefetches in a single trigger, thus improving timeliness. In these three traces, RICH's timeliness is improved by 35%, 30%, and 12% over Bingo, respectively.

5.3 Sensitivity to Increased Memory Latency

In view of the high-latency trend in the memory industry, we introduce additional latency for main memory and observe its impact on performance. Figure 13 shows the average performance improvements of RICH over the Bingo and PMP. As memory latency increases, the performance gap widens: the gap grows to 8.3% over Bingo and 6.2% over PMP at 120 ns additional latency. This trend is attributed to RICH's superior coverage and timeliness, as it can eliminate more cache misses. In systems with longer memory latency, these cache misses become increasingly detrimental to overall performance. These results suggest that our prefetcher is well-suited for future memory architectures characterized by long latency and high bandwidth. Essentially, RICH can accommodate a broader range of memory access patterns while leveraging high bandwidth to mitigate the adverse effects of long memory latency on system performance.

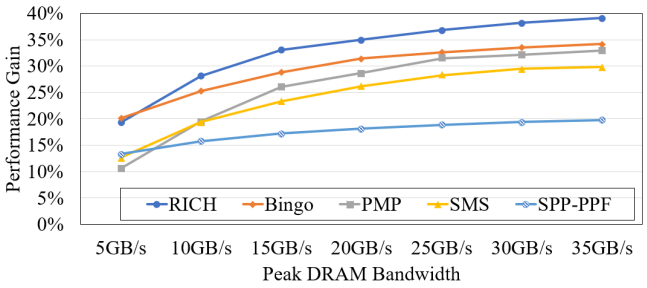


Figure 14: Performance under different memory bandwidth

5.4 Sensitivity to Memory Bandwidth

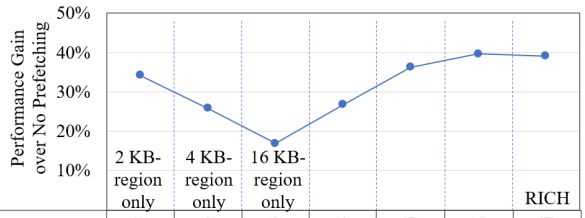
Figure 14 illustrates the impact of bandwidth on prefetcher performance. Under varying bandwidth constraints, the RICH prefetcher shows a substantial performance increase: its performance gain rises from 19% at 5 GB/s to 39% at 35 GB/s—an improvement that far outperforms that of other prefetchers, with Bingo increasing from 20% to 34%. This marked enhancement is attributable to the unique architectural benefits of the RICH prefetcher. By leveraging abundant metadata, RICH more effectively detects the characteristic of programs and issues prefetch requests across multiple regions, thereby improving bandwidth utilization. Moreover, under high-bandwidth conditions, off-chip pattern requests no longer impose an noticeable bandwidth burden.

Conversely, when bandwidth is severely limited (5GB/s), RICH's performance falls slightly below that of Bingo (19.3% vs 20.1%). Under such constraints, RICH's aggressive prefetching competes for the already scarce memory bandwidth, causing many prefetch requests to be dropped and rendered ineffective. Moreover, the off-chip metadata load requests may encroach on regular demand memory accesses. As for PMP, the performance drop under limited bandwidth is likely due to its low prefetch accuracy. As bandwidth increases, however, the high coverage advantage of the PMP prefetcher becomes more apparent, allowing it to gradually catch up with Bingo. It is important to emphasize that RICH is designed for the high-bandwidth memory trend, while the performance in bandwidth-constrained scenarios is not our primary focus.

5.5 Performance Breakdown Analysis

To quantify the effectiveness of our prefetch strategy and the on-chip/off-chip cooperation strategy, we evaluate the following configurations shown in Figure 15: (1) **2 KB-region only**: traditional single-address/offset 2 KB-region prefetching used in Bingo [12]. (2)-(3): single-offset prefetching with **larger region sizes** of 4 KB and 16 KB, respectively. (4) **Naïve multi-region**: prefetching three regions together but without the multi-offset trigger mechanism nor the region arbitration. (5) **Multi-region multi-trigger**: adding the multi-offset trigger mechanism. (6) **All-on-chip**: adding the region arbitration upon the multi-region multi-offset prefetching, with the entire structure placed on-chip. (7) **RICH (proposed)**: the complete RICH configuration, with the on-chip/off-chip cooperation strategy.

By comparing (1)-(3) against (4), we see naively combining 3 regions together ends up being worse than prefetching 2 KB-region alone. After introducing our multi-offset trigger mechanism in (5),



	(1)	(2)	(3)	(4)	(5)	(6)	(7)
Multi-region	×	×	×	√	√	√	√
Multi-offset	×	×	×	×	√	√	√
Region arbitration	×	×	×	×	×	√	√
Use off-chip	×	×	×	×	×	×	√
On-chip overheads (KB)	22	10	137	174	174	175	47.3

Figure 15: Multi-offset trigger mechanism significantly benefits RICH, while offloading part of the 16 KB-Region patterns has limited performance impact.

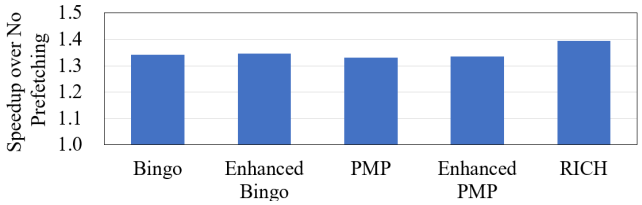


Figure 16: Iso-storage comparison shows RICH is more storage efficient than Bingo and PMP.

we retains the benefits of large-region prefetching while reducing the risk of misprediction. However, as unnecessary prefetches of 2 KB and 4 KB regions can overlap with 16 KB ranges in (5), less accurate prefetches may occur, leaving room for performance improvement. Our region arbitration added in (6) exactly addresses this issue. It blocks prefetches if a memory space has been covered by a more suitable region size. Finally, the full RICH design (7), with part of the 16 KB-region patterns outsourced to off-chip, suffers only a 0.5% performance loss compared to the fully on-chip configuration (6). As discussed in Section 3, the 16 KB-region patterns exhibit high tolerance to latency, placing them off-chip leads to only a small loss of prefetching opportunities. This result demonstrate the effectiveness of our on-chip/off-chip cooperation strategy.

5.6 Iso-storage Comparison

We demonstrate the storage efficiency of RICH through a comparison under similar storage budgets. We scale up the PHT of Bingo to 193 KB, referred to as *Enhanced Bingo*. *Enhanced PMP* uses 194 KB on-chip area. RICH remains its on-chip requirement of 47 KB with a main memory space of 128 KB, which are 175 KB in total. Note RICH uses far less precious on-chip area than others. As shown in Figure 16, RICH still holds substantial performance advantages against *Enhanced Bingo* and *Enhanced PMP*. This suggests that RICH is more capable of converting the abundant metadata into performance gains. By contrast, the additional metadata budgets bring limited benefits of 0.3% for Bingo and PMP. This aligns with our assumption that simple scaling-up of existing prefetchers faces the

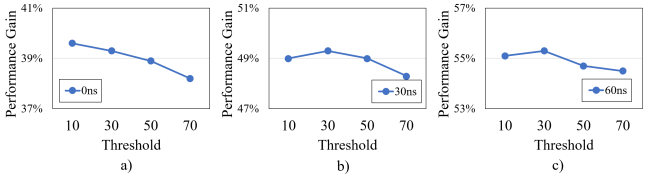


Figure 17: Sensitivity to different prefetch count thresholds for 16 KB-region under different memory latencies

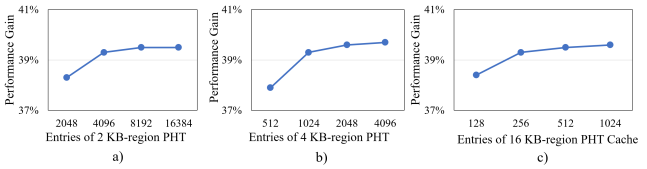


Figure 18: Sensitivity to different PHT sizes

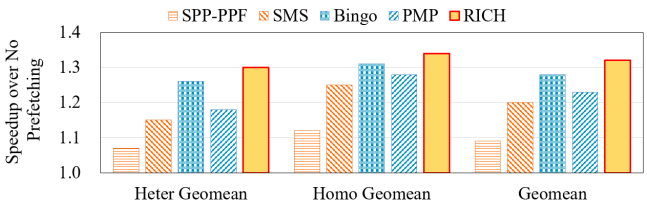


Figure 19: RICH outperforms Bingo in both homogeneous and heterogeneous workloads on a 4-core system

law of diminishing returns. In this experiment, we enlarge Bingo by increasing the associativity of its PHT to 25 ways. For PMP, we follow a similar scaling method used in its original paper [13]. Specifically, the trigger offset is configured to 12 bits and the counter size to 6 bits.

5.7 Sensitivity to Configurations

The prefetch count threshold: We examine the impact of the prefetch count threshold for 16 KB-region patterns under varying latencies, as illustrated in Figures 17. With 30 ns or 60 ns latency increases, the optimal performance is achieved with a threshold of 30. Increasing the threshold further reduces prefetch opportunities, while decreasing it causes more row buffer misses, both leading to performance degradation. In the case of the conventional memory latency, the highest performance is obtained with a threshold of 10, as the performance loss due to latency is not as pronounced as in the higher latency scenarios.

The PHT sizes: We examine the impact of the PHT sizes for different regions (2 KB/4 KB/16 KB). As illustrated in Figures 18, the 2 KB-region PHT saturates at 4096 entries, with some performance loss at 2048 entries due to the dual-trigger mechanism. The 4 KB-region PHT achieves performance saturation with 1024 entries. The 16 KB-region on-chip PHT, which stores only 256 entries, shows a slight performance loss when reduced further to 128 entries. This slight loss is attributed to our well-designed hierarchical storage mechanism, which efficiently utilizes off-chip resources to store 16 KB patterns.

5.8 Multi-core Performance

Figure 19 illustrates the multi-core performance of the prefetchers. RICH outperforms Bingo by 2.8%, PMP by 7%, SMS by 10%, and SPP-PPF by 22%. Although RICH's aggressive prefetching strategy delivers substantial gains, it consumes a larger share of bandwidth resources in the 4-core system. As a result, the performance improvement is slightly lower compared to the single-core system.

6 Related Work

Some spatial prefetchers based on delta sequences [17, 18, 65] reduce redundancy by sharing stride patterns but require sequential address generation via recursive look-ahead [66, 67]. This limits instant bulk prefetching and scalability in next-gen high-bandwidth memory systems.

Temporal prefetching. While many temporal prefetchers [14–16, 68] employ off-chip memory, some proposals successfully reduce storage requirements [33, 69, 70].

Virtual or physical address. The issue of physical address discontinuity hinders large-region prefetching. A recent study [71] shows that modern systems heavily use 2 MB pages, and by notifying the prefetchers with page sizes, it is feasible to enable large-region prefetching with physical addresses. We leave the incorporation of this technique as future work.

Feedback-based tuning. A prefetcher can leverage more information to tune prefetch aggressiveness [71–73]. We believe these techniques are orthogonal to RICH as many of them can build on top of the current RICH design. We also leave them as future work.

More techniques. Load value prediction [74–76] forecasts memory addresses and preemptively dispatches requests to the L1d cache . While it effectively reduces latency, its tight coupling with the micro-architecture incurs high costs, and incorrect predictions can cause rollback overheads. Runahead execution leverages idle computational resources by speculatively executing future instructions [77–81]. This approach utilizes otherwise wasted cycles during memory accesses. Some prefetchers [82–84] aim to address irregular memory accesses in pointer-rich data structures. [85] proposes a methodology to classify application behaviors, enabling reasoning on applicability of specific prefetchers. [86] proposes a profile-guided software prefetching technique that improves prefetch timeliness by leveraging dynamic execution time information.

7 Conclusion

Our work is aimed at future memory systems characterized by higher bandwidth, larger capacity, and higher access latency. We focus on enhancing prefetcher performance by utilizing abundant metadata while keeping overheads minimal. We introduce a selection mechanism that combines the high-performance benefits of large regions with the low misprediction cost of small regions. We offload infrequent 16 KB-region patterns to main memory to reduce overall overheads. RICH adapts well to a high-latency future memory system, showing pronounced performance benefits of 8.3% over Bingo and 6.2% over PMP. In a conventional system, RICH still achieves a 3.4% speedup over Bingo.

References

[1] Andy Patrizio. 2018. *Facebook and Amazon are causing a memory shortage.* https://www.networkworld.com/article/965006/facebook-and-amazon-are-causing-a-memory-shortage.html

[2] John Ousterhout, Parag Agrawal, David Erickson, Christos Kozyrakis, Jacob Leverich, David Mazières, Subhasish Mitra, Aravind Narayanan, Guru Parulkar, Mendel Rosenblum, Stephen M. Rumble, Eric Stratmann, and Ryan Stutsman. 2010. The case for RAMClouds: scalable high-performance storage entirely in DRAM. *ACM SIGOPS Operating Systems Review* 43, 4 (2010), 92–105.

[3] Seok-Hee Lee. 2016. Technology scaling challenges and opportunities of memory devices. In *IEEE International Electron Devices Meeting (IEDM)*.

[4] Youngmoon Lee, Hasan Al Maruf, Mosharaf Chowdhury, Asaf Cidon, and Kang G Shin. 2022. Hydra: Resilient and highly available remote memory. In *USENIX FAST*.

[5] Ameen Akel, Adrian M. Caulfield, Todor I. Mollov, Rajesh K. Gupta, and Steven Swanson. 2011. Onyx: a Protoype Phase Change Memory Storage Array. In *Proc. of USENIX HotStorage*.

[6] Takayuki Kawahara. 2010. Scalable spin-transfer torque ram technology for normally-off computing. *IEEE Design & Test of Computers* 28, 1 (2010), 52–63.

[7] Simone Raoux, Geoffrey W Burr, Matthew J Breitwisch, Charles T Rettner, Y-C Chen, Robert M Shelby, Martin Salinga, Daniel Krebs, S-H Chen, H-L Lung, and C. H. Lam. 2008. Phase-change random access memory: A scalable technology. *IBM Journal of Research and Development* 52, 4.5 (2008), 465–479.

[8] Shimeng Yu and Pai-Yu Chen. 2016. Emerging Memory Technologies: Recent Trends and Prospects. *IEEE Solid-State Circuits Magazine* 8, 2 (2016). https://doi.org/10.1109/MSSC.2016.2546199

[9] Debendra Das Sharma. 2022. Compute Express Link®: An open industry-standard interconnect enabling heterogeneous data-centric computing. In *IEEE Symp. on High-Perf. Interconnects*. https://doi.org/10.1109/HOTI55740.2022.00017

[10] D. Gouk, M. Kwon, H. Bae, S. Lee, and M. Jung. 2023. Memory Pooling With CXL. *IEEE MICRO* (2023). https://doi.org/10.1109/MM.2023.3237491

[11] S. Somogyi, T.F. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos. 2006. Spatial Memory Streaming. In *ISCA*. https://doi.org/10.1109/ISCA.2006.38

[12] Mohammad Bakhshalipour, Mehran Shakerinava, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Bingo Spatial Data Prefetcher. In *IEEE HPCA*.

[13] Shizhi Jiang, Qiusong Yang, and Yiwei Ci. 2022. Merging Similar Patterns for Hardware Prefetching. In *IEEE/ACM MICRO*.

[14] Akanksha Jain and Calvin Lin. 2013. Linearizing irregular memory accesses for improved correlated prefetching. In *IEEE/ACM MICRO*.

[15] Hao Wu, Krishnendra Nathella, Dam Sunwoo, Akanksha Jain, and Calvin Lin. 2019. Efficient Metadata Management for Irregular Data Prefetching. In *ACM/IEEE ISCA*. 449–461.

[16] Thomas F Wenisch, Michael Ferdman, Anastasia Ailamaki, Babak Falsafi, and Andreas Moshovos. 2009. Practical off-chip meta-data for temporal memory streaming. In *IEEE HPCA*. 79–90.

[17] Jinchun Kim, Seth H Pugsley, Paul V Gratz, AL Narasimha Reddy, Chris Wilkerson, and Zeshan Chishti. 2016. Path Confidence Based Lookahead Prefetching. In *IEEE/ACM MICRO*.

[18] Agustín Navarro-Torres, Biswabandan Panda, Jesús Alastruey-Benedé, Pablo Ibáñez, Víctor Viñals-Yúfera, and Alberto Ros. 2022. Berti: an Accurate Local-delta Data Prefetcher. In *IEEE/ACM MICRO*. 975–991.

[19] Saugata Ghose, Tianshi Li, Nastaran Hajinazar, Damla Senol Cali, and Onur Mutlu. 2019. Demystifying Complex Workload–DRAM Interactions: An Experimental Study. *ACM Meas. Anal. Comput. Syst.*, Article 60 (2019). https://doi.org/10.1145/3366708

[20] Chengning Wang, Dan Feng, Wei Tong, Jingning Liu, Zheng Li, Jiayi Chang, Yang Zhang, Bing Wu, Jie Xu, Wei Zhao, Yilin Li, and Ruoxi Ren. 2019. Cross-point Resistive Memory: Nonideal Properties and Solutions. *ACM Trans. Des. Autom. Electron. Syst.* (2019). https://doi.org/10.1145/3325067

[21] Scott W. Fong, Christopher M. Neumann, and H.-S. Philip Wong. 2017. Phase-Change Memory—Towards a Storage-Class Memory. *IEEE Trans. on Electron Devices* (2017). https://doi.org/10.1109/TED.2017.2746342

[22] JEDEC. 2013. JESD235A: High Bandwidth Memory (HBM) DRAM.

[23] Milan Radulovic, Darko Zivanovic, Daniel Ruiz, Bronis R. de Supinski, Sally A. McKee, Petar Radojković, and Eduard Ayguadé. 2015. Another Trip to the Wall: How Much Will Stacked DRAM Benefit HPC?. In *ACM MEMSYS*. https://doi.org/10.1145/2818950.2818955

[24] Solmaz Salehian and Yonghong Yan. 2017. Evaluation of Knight Landing High Bandwidth Memory for HPC Workloads. In *ACM IA*^{3}. Article 10. https://doi.org/10.1145/3149704.3149766

[25] Hongjing Huang, Zeke Wang, Jie Zhang, Zhenhao He, Chao Wu, Jun Xiao, and Gustavo Alonso. 2022. Shuhai: A Tool for Benchmarking High Bandwidth Memory on FPGAs. *IEEE TC* 71, 5 (2022). https://doi.org/10.1109/TC.2021.3075765

[26] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In *IEEE/ACM MICRO*. https://doi.org/10.1145/3613424.3614256

[27] JEDEC. 2009. JESD79-3E: DDR3 SDRAM Standard.

[28] JEDEC. 2017. JESD79-4C: DDR4 SDRAM Standard.

[29] JEDEC. 2020. JESD79-5: DDR5 SDRAM Standard.

[30] Babak Falsafi and Thomas F Wenisch. 2022. *A Primer on Hardware Prefetching*. Springer Nature.

[31] Rahul Bera, Konstantinos Kanellopoulos, Anant Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. 2021. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *IEEE/ACM MICRO*. 1121–1137.

[32] Mohammad Bakhshalipour, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2018. Domino temporal data prefetcher. In *IEEE HPCA*. 131–142.

[33] Hao Wu, Krishnendra Nathella, Matthew Pabst, Dam Sunwoo, Akanksha Jain, and Calvin Lin. 2022. Practical Temporal Prefetching With Compressed On-Chip Metadata. *IEEE Trans. Comput.* 71, 11 (2022). https://doi.org/10.1109/TC.2021.3065909

[34] Michael Ferdman, Thomas F. Wenisch, Anastasia Ailamaki, Babak Falsafi, and Andreas Moshovos. 2008. Temporal instruction fetch streaming. In *IEEE/ACM MICRO*. 1–10.

[35] Michael Ferdman, Cansu Kaynak, and Babak Falsafi. 2011. Proactive instruction fetch. In *IEEE/ACM MICRO*.

[36] Cansu Kaynak, Boris Grot, and Babak Falsafi. 2013. SHIFT: Shared history instruction fetch for lean-core server processors. In *IEEE/ACM MICRO*.

[37] Cansu Kaynak, Boris Grot, and Babak Falsafi. 2015. Confluence: Unified instruction supply for scale-out servers. In *IEEE/ACM MICRO*. 166–177. https://doi.org/10.1145/2830772.2830785

[38] Ali Ansari, Fatemeh Golshan, Rahil Barati, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2023. MANA: Microarchitecting a Temporal Instruction Prefetcher. *IEEE Trans. Comput.* 72, 3 (2023). https://doi.org/10.1109/TC.2022.3176825

[39] T. Sherwood, S. Sair, and B. Calder. 2000. Predictor-directed Stream Buffers. In *Proc. of IEEE/ACM MICRO*. https://doi.org/10.1145/360128.360135

[40] Suleyman Sair, Timothy Sherwood, and Brad Calder. 2003. A Decoupled Predictor-Directed Stream Prefetching Architecture. *IEEE Trans. Comput.* 52, 3 (2003).

[41] Mohammad Bakhshalipour, Seyedali Tabaeiaghdaei, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Evaluation of Hardware Data Prefetchers on Server Processors. *ACM Computing Surveys (CSUR)* 52, 3 (2019), 1–29.

[42] Mohammad Bakhshalipour, Pejman Lotfi-Kamran, Abbas Mazloumi, Farid Samandi, Mahmood Naderan-Tahan, Mehdi Modarressi, and Hamid Sarbazi-Azad. 2018. Fast Data Delivery for Many-Core Processors. *IEEE Trans. on Computers* 67, 10 (2018), 1416–1429.

[43] Pierre Michaud. 2016. Best-offset Hardware Prefetching. In *IEEE HPCA*. https://doi.org/10.1109/HPCA.2016.7446087

[44] Rahul Bera, Anant V. Nori, Onur Mutlu, and Sreenivas Subramoney. 2019. DSPatch: Dual Spatial Pattern Prefetcher. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*.

[45] Manjunath Shevgoor, Sahil Koladiya, Rajeev Balasubramonian, Chris Wilkerson, Seth H Pugsley, and Zeshan Chishti. 2015. Efficiently prefetching complex address patterns. In *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*.

[46] Naveen Muralimanohar, Rajeev Balasubramonian, and Norm Jouppi. 2007. Optimizing NUCA Organizations and Wiring Alternatives for Large Caches with CACTI 6.0. In *IEEE/ACM MICRO*.

[47] Nathan Gober, Gino Chacon, Lei Wang, Paul V Gratz, Daniel A Jimenez, Elvira Teran, Seth Pugsley, and Jinchun Kim. 2022. The championship simulator: Architectural simulation for education and competition. *arXiv preprint arXiv:2210.14324* (2022).

[48] [n. d.]. The Champsim Simulator. https://github.com/ChampSim/ChampSim

[49] 2015. The 2nd Data Prefetching Championship. https://comparch-conf.gatech.edu/dpc2/

[50] 2019. The 3rd Data Prefetching Championship. https://dpc3.compas.cs.stonybrook.edu/

[51] 2017. The 2nd Cache Replacement Championship. https://crc2.ece.tamu.edu/

[52] Andrei Frumusanu Ian Cutress. 2021. *Golden Cove Microarchitecture (P-Core) Examined.* https://www.anandtech.com/show/16881/a-deep-dive-into-intels-alder-lake-microarchitectures/3

[53] Chester Lam. 2021. *Popping the Hood on Golden Cove.* https://chipsandcheese.com/p/popping-the-hood-on-golden-cove

[54] Chester Lam. 2022. *Going Armchair Quarterback on Golden Cove's Caches.* https://chipsandcheese.com/p/going-armchair-quarterback-on-golden-coves-caches

[55] Andrei Frumusanu Ian Cutress. 2021. *Intel Architecture Day 2021: Alder Lake, Golden Cove, and Gracemont Detailed.* https://www.anandtech.com/show/16881/a-deep-dive-into-intels-alder-lake-microarchitectures/3

[56] Rahul Bera, Konstantinos Kanellopoulos, Shankar Balachandran, David Novo, Ataberk Olgun, Mohammad Sadrosadat, and Onur Mutlu. 2022. Hermes: Accelerating Long-latency Load Requests via Perceptron-based Off-chip Load Prediction. In *IEEE/ACM MICRO*.

[57] Eshan Bhatia, Gino Chacon, Seth Pugsley, Elvira Teran, Paul V Gratz, and Daniel A Jiménez. 2019. Perceptron-based prefetch filtering. In *Proc. of International Symposium on Computer Architecture*.

[58] 2006. SPEC CPU 2006. https://www.spec.org/cpu2006/
[59] 2017. SPEC CPU 2017. https://www.spec.org/cpu2017/
[60] Julian Shun and Guy E Blelloch. 2013. Ligra: a Lightweight Graph Processing Framework for Shared Memory. In *Proc. of ACM symposium on Principles and Practice of Parallel Programming*.
[61] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC Benchmark Suite: Characterization and Architectural Implications. In *Proc. of PACT*. 72–81. https://doi.org/10.1145/1454115.1454128
[62] [n. d.]. *DPC-3 Traces*. https://dpc3.compas.cs.stonybrook.edu/champsim-traces/
[63] SAFARI Research Group. 2024. *Ligra Traces for ChampSim*. https://doi.org/10.5281/zenodo.14267978
[64] SAFARI Research Group. 2024. *Parsec Traces for ChampSim*. https://doi.org/10.5281/zenodo.14268118
[65] Mehran Shakerinava, Mohammad Bakhshalipour, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Multi-lookahead offset prefetching. *The Third Data Prefetching Championship* (2019).
[66] Philippos Papaphilippou, Paul HJ Kelly, and Wayne Luk. 2019. Pangloss: a novel Markov chain prefetcher. *arXiv preprint arXiv:1906.00877* (2019).
[67] Shizhi Jiang, Yiwei Ci, Qiusong Yang, and Mingshu Li. 2021. Matryoshka: A Coalesced Delta Sequence Prefetcher. In *Proceedings of the 50th International Conference on Parallel Processing*. 1–11.
[68] Tingji Zhang, Boris Grot, Wenjian He, Yashuai Lv, Peng Qu, Fang Su, Wenxin Wang, Guowei Zhang, Xuefeng Zhang, and Youhui Zhang. 2025. Hierarchical Prefetching: A Software-Hardware Instruction Prefetcher for Server Applications. In *Proc. of ASPLOS*. 529–544. https://doi.org/10.1145/3676641.3716260
[69] Hao Wu, Krishnendra Nathella, Joseph Pusdesris, Dam Sunwoo, Akanksha Jain, and Calvin Lin. 2019. Temporal Prefetching Without the Off-Chip Metadata. In *IEEE/ACM MICRO*. 996–1008. https://doi.org/10.1145/3352460.3358300
[70] Sam Ainsworth and Lev Mukhanov. 2024. Triangel: A High-performance, Accurate, Timely on-chip Temporal Prefetcher. In *ACM/IEEE ISCA*.
[71] Georgios Vavouliotis, Gino Chacon, Lluc Alvarez, Paul V. Gratz, Daniel A. Jiménez, and Marc Casas. 2022. Page Size Aware Cache Prefetching. In *IEEE/ACM MICRO*. 956–974. https://doi.org/10.1109/MICRO56248.2022.00070
[72] Georgios Vavouliotis, Marti Torrents, Boris Grot, Kleovoulos Kalaitzidis, Leeor Peled, and Marc Casas. 2025. To Cross, or Not to Cross Pages for Prefetching?. In *IEEE HPCA*. 188–203. https://doi.org/10.1109/HPCA61900.2025.00025
[73] Santhosh Srinath, Onur Mutlu, Hyesoon Kim, and Yale N. Patt. 2007. Feedback Directed Prefetching: Improving the Performance and Bandwidth-Efficiency of Hardware Prefetchers. In *IEEE 13th International Symposium on High Performance Computer Architecture*.
[74] Rami Sheikh, Harold W Cain, and Raguram Damodaran. 2017. Load value prediction via path-based address prediction: Avoiding mispredictions due to conflicting stores. In *Proc. of IEEE/ACM MICRO*.
[75] Rami Sheikh and Derek Hower. 2019. Efficient load value prediction using multiple predictors and filters. In *IEEE HPCA*. 454–465.
[76] Sudhanshu Shukla, Sumeet Bandishte, Jayesh Gaur, and Sreenivas Subramoney. 2022. Register file prefetching. In *Proc. of ISCA*. 410–423. https://doi.org/10.1145/3470496.3527398
[77] James Dundas and Trevor Mudge. 1997. Improving data cache performance by pre-executing instructions under a cache miss. In *Proc. of International Conference on Supercomputing*. 68–75.
[78] Ajeya Naithani, Josué Feliu, Almutaz Adileh, and Lieven Eeckhout. 2020. Precise Runahead Execution. In *IEEE HPCA*. 397–410.
[79] Onur Mutlu, Hyesoon Kim, and Yale N Patt. 2005. Techniques for efficient processing in runahead execution engines. In *ISCA*. IEEE.
[80] Milad Hashemi and Yale N Patt. 2015. Filtered runahead execution with a runahead buffer. In *IEEE/ACM MICRO*. https://doi.org/10.1145/2830772.2830812
[81] Milad Hashemi, Onur Mutlu, and Yale N Patt. 2016. Continuous runahead: Transparent hardware acceleration for memory intensive workloads. In *IEEE/ACM MICRO*. https://doi.org/10.1109/MICRO.2016.7783764
[82] Feng Xue, Chenji Han, Xinyu Li, Junliang Wu, Tingting Zhang, Tianyi Liu, Yifan Hao, Zidong Du, Qi Guo, and Fuxin Zhang. 2024. Tyche: An Efficient and General Prefetcher for Indirect Memory Accesses. *ACM Trans. Archit. Code Optim.* 21, 2, Article 30 (March 2024), 26 pages.
[83] Xiangyao Yu, Christopher J. Hughes, Nadathur Satish, and Srinivas Devadas. 2015. IMP: Indirect memory prefetcher. In *IEEE/ACM MICRO*. 178–190.
[84] Heiner Litz, Grant Ayers, and Parthasarathy Ranganathan. 2022. CRISP: critical slice prefetching. In *Proc. of ACM ASPLOS*. 300–313. https://doi.org/10.1145/3503222.3507745
[85] Grant Ayers, Heiner Litz, Christos Kozyrakis, and Parthasarathy Ranganathan. 2020. Classifying Memory Access Patterns for Prefetching. In *Proc. of ASPLOS*. https://doi.org/10.1145/3373376.3378498
[86] Saba Jamilan, Tanvir Ahmed Khan, Grant Ayers, Baris Kasikci, and Heiner Litz. 2022. APT-GET: profile-guided timely software prefetching. In *Proc. of the European Conference on Computer Systems*. 747–764. https://doi.org/10.1145/3492321.3519583